ANH Extra documentation -Services

blacklotous - RKW Ltd.

May 2024

Contents

1 Introduction **2**

2 Target Theory **2**
2.1 Choosing Targets 2
2.2 How many targets? 2
2.3 visualization 3

3 Optimal Setup Calculation Methods **3**
3.1 Nestors' Random Combs 3
3.2 Black Lotous' Random Prices 4
3.3 Comparison between these 2 4
3.4 What's being used now as a best combs index 5

4 Optimal Coins Service **5**

1 Introduction

2 Target Theory

2.1 Choosing Targets

After being clear, that leverage play heavily changes fees, and is something to be avoid, our interest shifted to dynamic targets. These targets change dynamically, to always adapt on the prices' curve, trying to avoid the extreme case and keep the run contained to returns to open price or returns to first target case –gleichzeitig never slowing it down too much. Another question raised is which takeouts should be used on each target, and by extension which combination of targets and takouts would have given the best profit till now? Should we follow it? Below follows mathematical theory about how many targets and takeouts we can afford to have, and how to choose them.

2.2 How many targets?

We can declare here (**never applying or calculating with fees**, as they are a standard addition relying on how many trades, and their optimization lies on reducing the trades) a general formula for our profit at each target. Or better said, our profit at each target **case**. That implies the profit we get after hitting a target and returning to a previous to close!
Below we symbolize targets with x, and their corresponding takeouts with y. **It's mandatory and self explained that they share the same length!** For both algorithms below (second on NHC covers every NHC algorithm using dynamic takoeuts for the opposite position) a profit function P(n) (**n being the number of the targets claimed/hit**) is declared as –**Note: both algorithms use the EEA attribute**, exiting upon returning to previous target:

profit at n th target for **ENH** is:

$$P(n) = (\sum_{k=0}^{n} X_k * Y_k) - X_{n-1} + (100 - (\sum_{k=n-1}^{all} Y_k)) * X_{n-1} \tag{1}$$

profit at worst case target for **ENH** is:

$$P(n) = (\sum_{k=0}^{all} X_k * Y_k) - 100 * X_{last} \tag{2}$$

profit at returns to open case target for **ENH** is:

$$P(n) = X_0 * Y_0 \tag{3}$$

note that all should be devided by 100

profit at n th target for **NHC** is:

$$P(n) = (\sum_{k=0}^{n} X_k * Y_k) - (\sum_{k=0}^{n} X_k * Y1_k) + (100 - (\sum_{k=n-1}^{all} Y_k) * X_{n-1} - (100 - (\sum_{k=n-1}^{all} Y1_k)) * X_{n-1} \tag{4}$$

profit at worst case target for **NHC** is:

$$P(n) = (\sum_{k=0}^{all} X_k * Y_k) - (\sum_{k=0}^{all} X_k * Y1_k) \tag{5}$$

given that all but the last y is greater than the corresponding y1.

profit at returns to open case target for **NHC** is:

$$P(n) = X_0 * Y_0 - X_0 * Y1_0 \tag{6}$$

2.3 visualization

Please **NOTE the FOLLOWING**. For the below graphs, target and takeouts combinations are sorted using basic multyarray sort. It starts with the lowest first column, then the lowest second, and so on.
The photos can be found acompaning this pdf in discord
The number following their name ("tartak") symbolizes how many targets where used, and the number following the underscore symbolizes the case he graph studies!
For example, tartaks4-2 stands for, used 4 target profits and the case is returning to second target.

3 Optimal Setup Calculation Methods

Complexity Declaration Working on price list of length x IS O(a*X)
Working on targets n IS O(n)
Working on takeouts m IS O(m)

3.1 Nestors' Random Combs

Given a price list of length x, targets of length n, takeouts of length y –where length stands for the number of iterations a loop needs to cover them all– Nestor proposed:
For all prices (x) and for 10/100 of all combinations of targets and takeouts ((n*m)/10) calculate the profits. Select the higher profits and search around them, if they are not deemed enough.

So we get a total complexity of O((n*m)/10 * x) and after corresponding weights adjustments:
O(a*x * (n*m)/10

3.2 Black Lotous' Random Prices

We select 10/100 of the available points and use the as starting points, so x/10 prices. We proceed to calculate on them for all the combinations available (of targets and takeouts). So in general we have complexity of O(a*x/10 * (n*m)).

3.3 Comparison between these 2

To compare them we are gonna need a visual understanding, so we will do the following. For **GIVEN** prices we are gonna calculate the profit based on all targets and takeouts combinations and map it to the corresponding profits. So we have a 3-D function such as:

$$z = f(x, y) where (z, y, x) = (profits, takeouts, targets) \tag{7}$$

Note: 10/100 isn't static, can be anything Then we will show this function only on the x-y plane, using colours to simulate the z-x, z-y plan, like seeing a mountain from top, but instead of using circles and their density to show height and depth we use colours. The warmer the colour the higher the spike and the colder the colour the deeper the depth.
To understand the difference between the two algorithms, imagine that we have a square, that consists of prices. The prices define it. Nestors' algorithm proceeds to create it whole and select random points on it.
Black Lotous' algorithm creates fragments of the square. but checks it whole.
For demonstrative purposes, a picture will be provided, but the analogies aren't set correctly, it's just to imagine the 2 different solutions:

Result Both algorithms seem to show the same complexity.
To prove which one is better and more accurate, at least a thousand runs on different (but same as inputs in the 2 algorithms) set of prices should be used, and that's because they have random part (selecting where to check (first algorithm) and where to build (second algorithm)). To view the final distribution of accuracy, one must test them for a long time.
Personally I believe the second is better, because iterating through the one tenth of prices is faster (prices iterations hides some boolean operations). Then again what one algo lacks in speed makes up for it in accuracy, and vice versa. I really hope to get proven right one day (equal if not better accuracy and faster execution time for the second), though there's no time for this now.

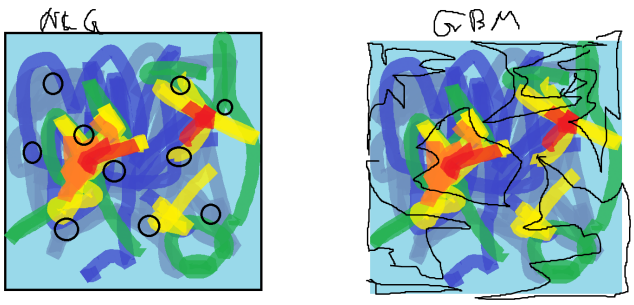

Figure 1: An example image.

3.4 What's being used now as a best combs index

Old service The first service we used as a heavy gunner. It used to iterate all prices, for all targets for all takeouts and force choose best settings on the past prices after seeing every possible outcome. That was slow as fuck. It was beobachtet, that its' execution time was O(t) where t was the price collection time!!!... The complexity though (without the time dilation problem) was O(x*m*n)

Itachi service This one aims o calculate first a weights parameter for the targets and prices. That says for all prices and targets –but not for all takeouts also– calculate the frequency of appearance of each target. Then it simply makes the calculation weighs*targets*takeouts to generate a profit index. This goes faster because it has complexity of O(x*m +m*n), breaking down the complexity, and speeding things up.

Future Itachi service A theoretical version, it should solely rely on the price wave of the market. Avoiding combinations calculation and relying on volatility, volume, market move and price curve move. Thoughts on this (**it's mandatory to share thoughts!**)?

4 Optimal Coins Service

SOS NOTE, NOT FOUND YET, PLEASE STUDY AEVO BEST CASES AND TODAYS' VELO +20/100 IN 4 HOURS. WE NEED SOME INDICATORS (MATHEMATICAL NOT TECHNICAL ANALYSIS STUFF) TO DETERMINE WHICH COINS TO USE EACH

TIME. IT'S MORE IMPORTANT THAN YOU THINK. SO THINK IT!
tILL WE MEET AGAIN...